More about Conditionals

CSL100

August 22

This Class

- More about if-else statements
- Switch-case
- Break

If-else Review

```
#include<stdio.h>
 int main() {
        int i = 5, j = 6, k = 7;
        if(i > j == k)
                printf("%d %d %d", i++, ++j, --k);
         else
                printf("%d %d %d", i, j, k);
         return 0;
 }
```

5 6
7

```
#include<stdio.h>
 int main() {
         int i = 5;
         if(i = i - 5 > 4)
                printf("inside if block");
         else
                printf("inside else block");
        return 0;
 }
```

Inside else block

If-else Review

```
#include<stdio.h>
 int main() {
            if(sizeof(0))
                        printf(“Yes");
            else
                        printf(“No");
            return 0;
 }
```

Yes

```
#include <stdio.h>
int main()
{
    char val=1;
    if(val--==0)
        printf("TRUE");
    else
        printf("FALSE");
    return 0;
}
```

FALSE

Unmatched if and else

```
if ((a !=    &&    !=
0))
    if (a * b    0)
        prin
(“positi    );
else
    printf(“zero”);
```

OUTPUT for a = 5, b = 0
NO OUTPUT!!
OUTPUT for a = 5, b = -5
zero

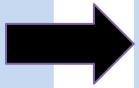
```
   ((a != 0) && (b !=
0))
    if (a * b >= 0)
        printf
(“positive”);
    else
```

OUTPUT for a = 5, b = 0
NO OUTPUT!!
OUTPUT for a = 5, b = -5
negative

Unmatched if and else

- An else always matches closest unmatched if
  - Unless forced otherwise using { … }

```
if (cond1)
    if (cond2)
        …
    else
        …
```

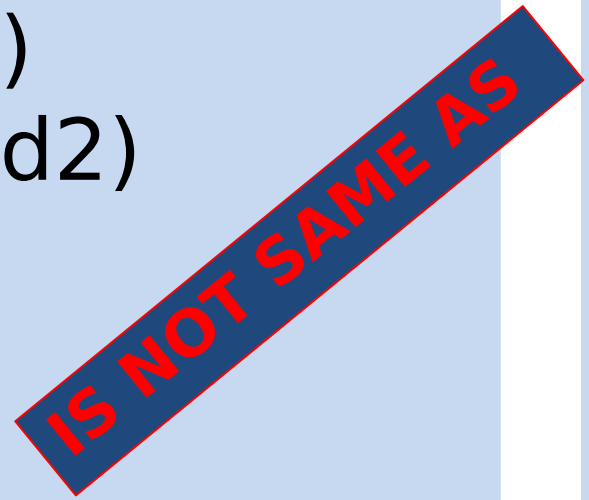


```
if (cond1) {
    if (cond2)
        …
    else
        …
}
```

Unmatched if and else

- An else always matches closest unmatched if
  - Unless forced otherwise using { … }

```
if (cond1)
    if (cond2)
        …
    else
        …
```




```
if (cond1) {
    if (cond2)
        …
}
else
        …
```

Else if

- A special kind of nesting is the chain of if-else-if-else-… statements

```
if (cond1) {
        stmt1
} else {
       if (cond2)  {
           stmt2
       } else {
          if (cond3) {
             ....
          }
       }
}
```

General form of if-else-if-else…

```
if (cond1)
        stmt-block1
else if (cond2)
    stmt-block2
else if (cond3)
    stmt-block3
else if (cond4)
    stmt-block4
else if  …
else
     last-block-of-
stmt
```

Example

- Given an integer , where , print the name of the weekday corresponding to .

  1: Sunday

  2: Monday

  ...

  7: Saturday

Printing the day

```
int day;
scanf (“%d”, &day);
if (day == 1) { printf(“Sunday”); }
else if (day == 2) { printf (“Monday”); }
else if (day == 3) { printf (“Tuesday”); }
else if (day == 4) { printf (“Wednesday”); }
else if (day == 5) { printf (“Thursday”); }
else if (day == 6) { printf (“Friday”); }
else if (day == 7) { printf (“Saturday”); }
else { printf (“ Illegal day %d”, day); }
```

Example 2

- Given an integer , where , print Weekday, if the corresponds to weekday, print Weekend otherwise.

  1, 7: Weekend

  2,3,4,5,6: Weekday

Weekday - version 1

```
int day;
scanf (“%d”, &day);
if (day == 1) { printf(“Weekend”); }
else if (day == 2) { printf (“Weekday”); }
else if (day == 3) { printf (“Weekday”); }
else if (day == 4) { printf (“Weekday”); }
else if (day == 5) { printf (“Weekday”); }
else if (day == 6) { printf (“Weekday”); }
else if (day == 7) { printf (“Weekend”); }
else { printf (“ Illegal day %d”, day); }
```

Weekday - version 2

```
int day;
scanf (“%d”, &day);
if ((day == 1) || (day == 7)) {
      printf(“Weekend”);
} else if (  (day == 2) || (day == 3)
          || (day == 4) || (day == 5)
          || (day == 6)) {
      printf (“Weekday”);
} else {
      printf (“ Illegal day %d”, day);
}
```

Weekday - version 3

```
int day;
scanf (“%d”, &day);
if ((day == 1) || (day == 7)) {
        printf(“Weekend”);
} else if ( (day >= 2) && (day <= 6) ) {
        printf (“Weekday”);
} else {
        printf (“ Illegal day %d”, day);
}
```

Switch-Case Statement

- Multi-way decision
- Checks whether an expression matches one out of a number of constant integer values
- Execution *branches* based on the match found

Printing the day, version 2

```
switch (day) {
case 1: printf(“Sunday”); break;
case 2: printf (“Monday”); break;
case 3: printf (“Tuesday”); break;
case 4: printf (“Wednesday”); break;
case 5: printf (“Thursday”); break;
case 6: printf (“Friday”); break;
case 7: printf (“Saturday”); break;
default: printf (“ Illegal day %d”, day);
}
```

Weekday, version 4

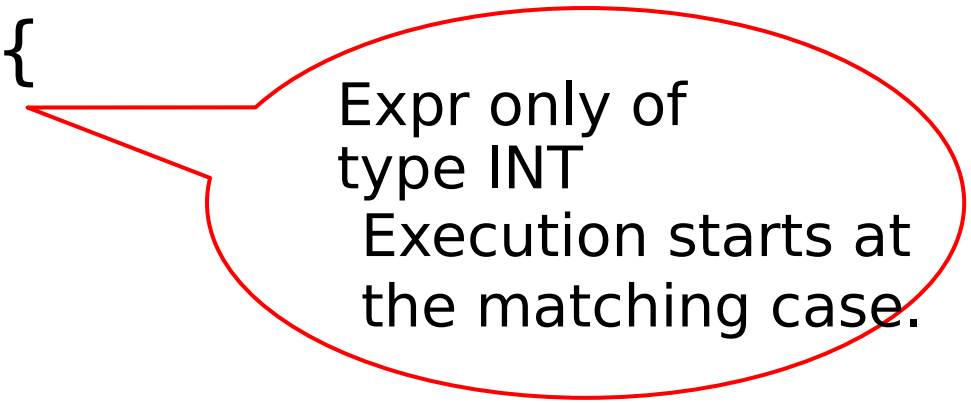
```
switch (day) {
case 1:
case 7: printf (“Weekend”); break;
case 2:
case 3:
case 4:
case 5:
case 6: printf (“Weekday”); break;
default: printf (“ Illegal day %d”, day);
}
```

The *break* Statement

- Syntax –
  - break;
- Used to break out of current code branch
  - Not to end the program

General Form of switch-case

```
switch (selector-expr) {
case label1: s1; break;
case label2: s2; break;
...
case labelN: sN; break;
default : sD;
}
```



- **default** is optional. (= *remaining cases*)
- The location of **default** does not matter.
- The statements following a case label are executed one after other

Example

- Switch expressions can only be of type *int*

```
#include<stdio.h>
 int main() {
          char ch = 65;
          switch(ch) {
                    case 'A': printf("Apple");
                    break;
                    case 'B': printf("Bing");
                    break;
                   default: printf("Bye");
                    break;
           }
          return 0;
 }
```

```
Appl
e
```

Fall Through…

```
int n = 100;
int digit = n%10; // last digit
switch (digit) {
default : printf("Not divisible by 5\n");
          break;
case 0: printf("Even\n");
case 5: printf("Divisible by 5\n");
        break;
}
```

What is printed by the program fragment?

Answer:
Even
Divisible by 5;

Class Quiz

- What is the value of expression:

(5<2) && (3/0)

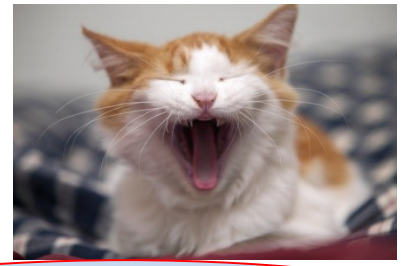
a) Compile time error



b) Run time crash



c) I don’t know / I don’t care

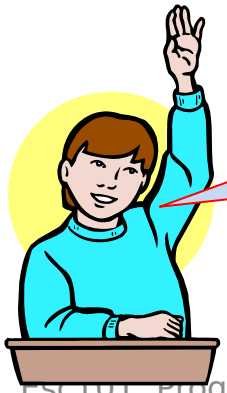


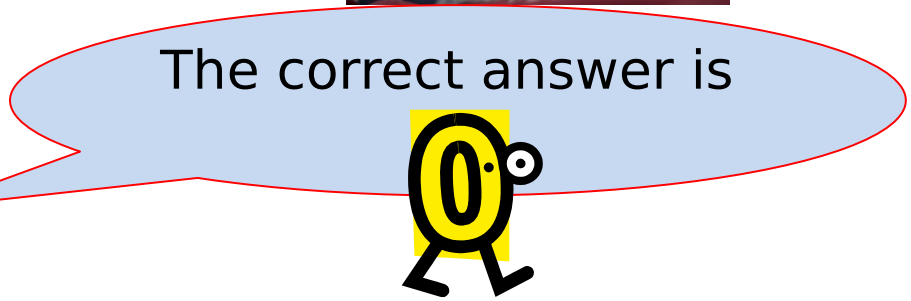
d) 0



e) 1

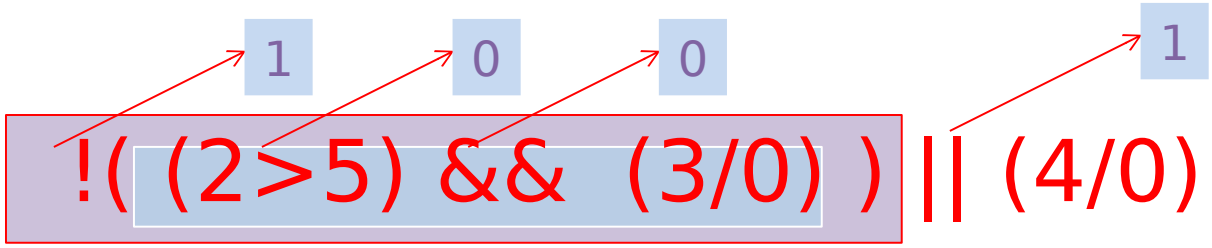






Short-Circuit Evaluation

- Do not evaluate the second operand of binary logical operator if result can be deduced from first operand
  - Arguments of && and || are evaluated from left to right (in sequence)
  - Also applies to nested logical operators



Evaluates to 1

Next Class

- Loops (6.3-6.5)
  - While statement
  - Do-while statement
  - For statement